Student Guide: Autoscaling with AWS Lambda

Nov 5, 2026 · @Bayu Samudra

What you'll do

You'll package a small app as a container image, run it on AWS Lambda, and then send traffic to it with k6 to watch Lambda create copies (replicas) of it automatically.

The app gives every replica a random ID called `env_id`. By counting how many different IDs come back, you can see exactly how many replicas Lambda is running.

By the end you should be able to:

1. Build a Lambda container image, push it to Amazon ECR and deploy it.
2. Explain when Lambda reuses a replica and when it creates a new one.
3. Predict how many replicas a burst of users needs, and explain why the first requests are slower.

Work in the **Asia Pacific (Jakarta)** Region, `ap-southeast-3`, for every step.

Rule: write your prediction in the results sheet *before* you run the experiment.

Before you start

Tick each item before class.

- [ ] AWS account access. On AWS Academy Learner Lab, you'll use the existing **LabRole** instead of creating a role.
- [ ] Jakarta Region enabled: account menu (top right) → **Account** → **AWS Regions** → Asia Pacific (Jakarta). If it says **Disabled**, click **Enable** and wait a few minutes.
- [ ] AWS CLI v2 installed and configured. `aws sts get-caller-identity` prints your account number.
- [ ] Docker installed and running. `docker version` works. No Docker? Pair with a classmate, or use the zip option in Part 1.
- [ ] k6 installed. `k6 version` works.
  - macOS: `brew install k6`
  - Windows: `winget install k6 --source winget`

- Linux: use the apt or dnf package from the k6 install docs

- [ ] A folder named `lambda-scale` containing the three files from the next section.
- [ ] Windows users: run the commands in **Git Bash** or **WSL**, so the `bash` commands in this guide work as written.

The three files

Create these files in your `lambda-scale` folder.

app.py

Each request keeps its replica **busy** for `busy` seconds. A replica handles one request at a time, so while it's busy, the next request needs a different replica. The code at the top of the file runs once when a replica starts; `INIT_DELAY` makes that startup take a little time on purpose.

```
import json, os, time, uuid

# ---- Init phase: runs once per replica (execution environment) ----
INIT_TYPE = os.environ.get("AWS_LAMBDA_INITIALIZATION_TYPE", "on-demand")
time.sleep(float(os.environ.get("INIT_DELAY", "0")))   # simulate slow
startup
ENV_ID = uuid.uuid4().hex[:8]                              # one ID per replica
invocations = 0

MAX_BUSY = 5   # seconds; keep it below the function timeout


def handler(event, context):
    global invocations
    invocations += 1
    qs = event.get("queryStringParameters") or {}
    busy = min(float(qs.get("busy", 1)), MAX_BUSY)

    start = time.perf_counter()
    time.sleep(busy)   # hold this replica: it can't take another request
until done
    duration_ms = round((time.perf_counter() - start) * 1000)

    return {
        "statusCode": 200,
```

```
        "headers": {"Content-Type": "application/json"},
        "body": json.dumps({
            "env_id": ENV_ID,
            "init_type": INIT_TYPE,
            "invocations_in_this_env": invocations,
            "busy_s": busy,
            "duration_ms": duration_ms,
        }),
    }
```

Response field	Meaning
`env_id`	ID of the replica that served your request
`init_type`	`on-demand`: the replica was started when a request needed it
`invocations_in_this_env`	How many requests this replica has served
`busy_s`	How long the request held the replica
`duration_ms`	Measured busy time

Dockerfile

```
FROM public.ecr.aws/lambda/python:3.12
COPY app.py ${LAMBDA_TASK_ROOT}
CMD ["app.handler"]
```

k6.js

A k6 virtual user (VU) sends a request, waits for the answer, then sends the next one. With `busy=1`, each VU keeps about one request in progress, so **requests in progress ≈ number of VUs**.

```
import http from 'k6/http';
import { check, sleep } from 'k6';
import { Counter, Rate } from 'k6/metrics';

const envHits   = new Counter('env_hits');      // tagged with env_id and
init_type
const throttled = new Rate('throttled');        // share of 429 responses
```

```
const status200 = new Counter('status_200');    // OK
const status429 = new Counter('status_429');    // Too Many Requests
(throttled)
const statusErr = new Counter('status_other');  // anything else: 403, 5xx, 0
= network error

const URL  = __ENV.URL;
const BUSY = __ENV.BUSY || '1';                 // seconds each request holds
a replica
const VUS  = Number(__ENV.VUS || 50);

const scenarios = {
  warm:  { executor: 'shared-iterations', vus: 1, iterations: 10 },
  burst: { executor: 'shared-iterations', vus: VUS, iterations: VUS * 2 },
};

export const options = {
  scenarios: { main: scenarios[__ENV.SCENARIO || 'burst'] },
};

export default function () {
  const res = http.get(`${URL}?busy=${BUSY}`);
  throttled.add(res.status === 429);
  check(res, { 'status is 200': (r) => r.status === 200 });

  if (res.status === 200) {
    status200.add(1);
    const body = res.json();
    envHits.add(1, { env_id: body.env_id, init_type: body.init_type });
  } else if (res.status === 429) {
    status429.add(1);
    sleep(1);   // back off like a well-behaved client
  } else {
    statusErr.add(1, { status: String(res.status) });
    console.warn(`HTTP ${res.status}: ${String(res.body).slice(0, 120)}`);
  }
}
```

Scenario	What it does
`warm`	1 user, 10 requests one after another

Scenario	What it does
`burst`	`VUS` users at once, 2 requests each

In the k6 summary, the **CUSTOM** section shows how many responses were `status_200`, `status_429` (throttled by Lambda) and `status_other`. Any other status is also printed as a warning while the test runs.

Part 1 · Build and deploy

Open a terminal in your `lambda-scale` folder and set these variables once:

```
REGION=ap-southeast-3                # Asia Pacific (Jakarta)
export AWS_DEFAULT_REGION=$REGION
ACCT=$(aws sts get-caller-identity --query Account --output text)
```

If you open a new terminal later, set them again.

Step 1 · Build the image

```
docker build --platform linux/amd64 --provenance=false -t lambda-scale .
```

✅ Check: `docker images` lists `lambda-scale`.

Step 2 · Test it on your laptop

```
docker run --rm -p 9000:8080 lambda-scale
```

In a second terminal:

```
curl -XPOST "http://localhost:9000/2015-03-31/functions/function/invocations"
\
  -d '{"queryStringParameters":{"busy":"1"}}'
```

✅ Check: the answer contains an `env_id`. Stop the container with Ctrl+C.

Step 3 · Push the image to ECR

Console:

1. Check that the Region selector (top right) shows **Asia Pacific (Jakarta)**.

2. Open **Amazon ECR** → **Private registry** → **Repositories** → **Create repository**.
3. Repository name: your own name, the same one you use for `<your-repository-name>` in the CLI → **Create**.
4. Open the repository → **View push commands**. Run the login, tag and push commands it shows, but use the tag `v1` instead of `latest`. Skip its build command; you already built the image.

CLI:

```
REPO=$ACCT.dkr.ecr.$REGION.amazonaws.com/<your-repository-name>
aws ecr create-repository --repository-name <your-repository-name>
aws ecr get-login-password | docker login --username AWS --password-stdin
$ACCT.dkr.ecr.$REGION.amazonaws.com
docker tag lambda-scale:latest $REPO:v1
docker push $REPO:v1
```

✅ Check: the `v1` tag appears in your repository.

Step 4 · Create the function

Console:

1. Open **AWS Lambda** → **Functions** → **Create function**.
2. Choose **Container image**. Name: `lambda-scale`.
3. Container image URI: **Browse images** → your repository → tag `v1` → **Select image**.
4. Expand **Additional settings** and check these three:
   - **ARM64 architecture**: leave it **off**, so the function stays x86_64 like your image.
   - **Custom execution role**: on Learner Lab, turn it **on** and choose `LabRole`. Otherwise leave it off and Lambda creates a basic role.
   - **Function URL**: leave it **off**. You create it in Step 5.
5. Click **Create function** and wait about a minute.
6. **Configuration** → **General configuration** → **Edit** → Timeout **10 sec** → **Save**.
7. **Configuration** → **Environment variables** → **Edit** → add key `INIT_DELAY`, value `2` → **Save**.

CLI (on Learner Lab, skip the two `iam` commands and use the LabRole ARN):

```
cat > trust.json <<'EOF'
{"Version":"2012-10-17","Statement":[{"Effect":"Allow","Principal":
{"Service":"lambda.amazonaws.com"},"Action":"sts:AssumeRole"}]}
EOF
aws iam create-role --role-name lambda-scale-role --assume-role-policy-
document file://trust.json
aws iam attach-role-policy --role-name lambda-scale-role \
  --policy-arn arn:aws:iam::aws:policy/service-
role/AWSLambdaBasicExecutionRole
ROLE_ARN=arn:aws:iam::$ACCT:role/lambda-scale-role   # Learner Lab:
arn:aws:iam::$ACCT:role/LabRole

aws lambda create-function --function-name lambda-scale \
  --package-type Image --code ImageUri=$REPO:v1 \
  --role $ROLE_ARN --architectures x86_64 --timeout 10 \
  --environment "Variables={INIT_DELAY=2}"
```

If it says the role can't be assumed, wait 10 seconds and run `create-function` again.

Step 5 · Function URL

Console:

1. On the function page: **Configuration** → **Function URL** → **Create function URL**.
2. Auth type **NONE** → **Save**. The console also adds the permission that lets anyone call the URL.
3. Copy the URL and save it in your terminal without the final slash:

```
URL=https://xxxxxxxx.lambda-url.ap-southeast-3.on.aws
```

CLI:

```
aws lambda create-function-url-config --function-name lambda-scale --auth-
type NONE
aws lambda add-permission --function-name lambda-scale \
  --statement-id UrlPublicInvokeUrl --action lambda:InvokeFunctionUrl \
  --principal "*" --function-url-auth-type NONE
aws lambda add-permission --function-name lambda-scale \
  --statement-id UrlPublicInvokeFunction --action lambda:InvokeFunction \
  --principal "*" --invoked-via-function-url
URL=$(aws lambda get-function-url-config --function-name lambda-scale --query
FunctionUrl --output text)
URL=${URL%/}
```

First call: open `<your URL>?busy=1` in a browser and refresh a few times.

✅ Check: the first load is slower than the next ones, and `env_id` stays the same. Paste your URL into the class sheet.

No Docker? Create the function with **Author from scratch**, runtime **Python 3.12**. Paste `app.py` into the code editor, set **Runtime settings** → **Handler** to `app.handler`, click **Deploy**, then do Step 4 items 6–7 and Step 5.

Watching replicas

Each replica writes to its own log stream, so counting log streams every 10 seconds tells you how many replicas were working. Set this up once and refresh it after each run.

1. Open **CloudWatch** → **Logs** → **Logs Insights**.
2. In **Select log groups**, choose `/aws/lambda/lambda-scale`.
3. Set the time range to **15m**.
4. Paste the query below → **Run query**.
5. Open the **Visualization** tab → choose **Line**.
6. Click **Save** so you can reopen it from **Queries**.

```
filter @type = "REPORT"
| stats count_distinct(@logStream) as active_replicas,
        count(*) as requests,
        avg(@duration) as avg_duration_ms
  by bin(10s)
| sort @timestamp asc
```

Column	Meaning
`active_replicas`	Replicas that served at least one request in that 10-second window
`requests`	Requests finished in that window
`avg_duration_ms`	Average time a request held its replica

Logs arrive a few seconds late. If the chart looks empty, wait 20 seconds and run the query again.

Optional live dashboard while k6 runs: add `K6_WEB_DASHBOARD=true` in front of a `k6 run` command and open `http://localhost:5665`.

The experiment · One user vs a burst

Question: when does Lambda create a new replica?

Predict: how many replicas for 1 user sending 10 requests in a row? For 50 users at the same time?

Run:

```
k6 run -e URL=$URL -e SCENARIO=warm k6.js            # 1a: 1 user, 10 requests
k6 run -e URL=$URL -e SCENARIO=burst -e VUS=50 k6.js  # 1b: 50 users, 100 requests
```

Observe and record:

- `active_replicas` in Logs Insights for each run.
- From the k6 summary: the median (`med`) and `p(95)` of `http_req_duration`.

Think about it:

1. 1a sent 10 requests but needed only one replica. Why?
2. 1b sent 100 requests. Why weren't 100 replicas created?
3. Why is p95 latency in 1b much higher than the median?

When you're done, do the cleanup in the next section.

Cleanup

Do this before you leave. Your Function URL is public: anyone with the link can call it until you delete the function.

Console:

1. **Lambda** → **Functions** → select `lambda-scale` → **Actions** → **Delete** → type `confirm` → **Delete**. This also removes the URL.
2. **ECR** → **Repositories** → select your repository → **Delete** → type `delete` → **Delete**.
3. **CloudWatch** → **Log groups** → select `/aws/lambda/lambda-scale` → **Actions** → **Delete log group(s)**.
4. If the console created a role for you: **IAM** → **Roles** → find `lambda-scale-role-...` → **Delete**.

CLI:

```
aws lambda delete-function --function-name lambda-scale
aws ecr delete-repository --repository-name <your-repository-name> --force
aws logs delete-log-group --log-group-name /aws/lambda/lambda-scale

# only if you created the role yourself (not on Learner Lab)
aws iam detach-role-policy --role-name lambda-scale-role \
  --policy-arn arn:aws:iam::aws:policy/service-
role/AWSLambdaBasicExecutionRole
aws iam delete-role --role-name lambda-scale-role
```

- [ ] Function deleted
- [ ] ECR repository deleted
- [ ] Log group deleted
- [ ] IAM role deleted (if you created one)

Results sheet

Fill the predicted column before you run.

Run	Setting	Predicted replicas	Measured replicas	Median latency	p95 latency
1a warm	1 user, 10 requests				
1b burst	50 users, 100 requests				

Autoscaling framework

Answer these five questions for Lambda today. You'll fill the second column in the next practicum (VMs with a load balancer and Auto Scaling).

Question	Lambda	VM + Auto Scaling
What is the scaling unit?		
What triggers scaling?		
How fast does new capacity appear?		
What is the limit?		
How does it scale in?		

Reflection (for your lab report)

1. In your own words, what makes Lambda create a new replica?
2. What surprised you most in the experiment, and why?
3. Pick one real app (for example a ticket-booking site on sale day). What happens to the first users when thousands arrive in the same second, and why?

Troubleshooting

Problem	Likely cause	Fix
Jakarta is missing from the Region list, or commands fail there	Jakarta Region not enabled	Account → AWS Regions → Asia Pacific (Jakarta) → Enable, then wait a few minutes
"image manifest media type not supported"	Image built without `--provenance=false`	Build again with the exact command in Step 1, then push again
Function fails with "exec format error"	Built on an Apple Silicon Mac without `--platform linux/amd64`	Build again with the exact command in Step 1

Problem	Likely cause	Fix
`docker push` says "denied" or "no basic auth credentials"	ECR login expired or wrong Region	Run the `get-login-password` command again
"Task timed out after 3.00 seconds"	Timeout not changed	Set timeout to 10 s (Step 4)
URL returns 403 Forbidden	Permissions missing (CLI route)	Run both `add-permission` commands in Step 5
k6 says `URL` is undefined	The `URL` variable isn't set in this terminal	Run `echo $URL`; set it again as in Step 5
Many 429 errors in the experiment	Your account allows fewer replicas than the test uses	Ask your instructor for smaller numbers
Logs Insights shows nothing	Logs are delayed, or wrong time range or log group	Wait 20 s, check the log group and widen the time range

Still stuck? Paste the exact error message into the class sheet next to your URL.